From a path check to a container

Sandbox · the ladder

The explainer's chapter 08 admitted the limit: file tools are contained, a shell is where the fence gets thin. This is every option for closing that gap, cheapest first — code, then the operating system, then Docker, then the tiers you only need once someone else's code runs on your machine.

7 tiers · tier 1 = 117 lines · tier 4 = docker run · 1 interface · 9 red-team checks · This page was sandbox.html; it is now sandbox-guide.html.

CONTENTS

What is in this explainer

01 · What a sandbox is for 3

02 · The ladder 4

03 · Tier 0 — the prompt 5

04 · Tier 1 — containment in code 6

05 · Tier 2 — process hardening 7

06 · Why hardening is not isolation 9

07 · Tier 3 — operating system 10

08 · Tier 4 — Docker 11

09 · What Docker does not give you 13

10 · Tier 5 — stronger kernel isolation 14

11 · Tier 6 — someone else's machine 15

12 · Wiring it into the agent 16

13 · Red-team it 17

14 · Which rung, when 18

Page numbers refer to the folio at the foot of each page.

01 · What a sandbox is for

Chapter 08 of the explainer ended on a warning: the file tools are contained, but `run_command` is a real shell and nothing stops it walking out of `workspace/`. That is the honest answer for a laptop demo, and it is also the point where the interesting engineering starts.

This page is the build-up in the order you would actually do it: **containment in code first**, because it is free and it teaches you what the threat is; then **process hardening**; then the operating system; then **Docker**, which is the step most teams take next; then the things that exist once a container is not enough. Each tier is described by what it stops, what it costs, and what it does *not* stop.

A container is not a magic word. It is one rung on a ladder, and the rungs below it are cheap.

QUESTION	HONEST ANSWER
What is being sandboxed?	Two different things: ***files*** the agent writes, and ***commands*** it runs. The second is the hard one.
Against whom?	A model that is wrong, not an attacker with shell access. Those are different threat models and they want different tiers.
What does “enough” mean?	Enough that a mistake costs a file, not your home directory, your API key, or the machine.

> Threat model first, mechanism second. A watched live demo has a human in the loop, which is already a control. An unattended loop at 2 a.m. has none, and that is the run that justifies a container.

02 · The ladder

Seven rungs, cheapest first. Each one is a superset of the one above it, and each one buys a specific property rather than “security” in general.

THE LADDER

```
tier 0  prompt + policy    asks nicely
      |
tier 1  path containment   files stay in workspace/
      |
tier 2  hardening          deny-list, timeouts, env
      |
tier 3  OS isolation       uid, namespaces, no net
      |
tier 4  container          own fs + own network
      |
tier 5  kernel isolation   gVisor / Kata / microVM
      |
tier 6  hosted sandbox     never on your machine
```

TIER	STOPS	DOES NOT STOP	COST TO ADOPT
0	Nothing. It models intent.	Any mistake at all	0 lines
1	`../../secrets`, absolute paths, junctions	Any command	Already done
2	The eight catastrophic patterns; runaway output; hangs	An inventive command	One afternoon
3	Reading the rest of the filesystem, network egress, uid-based damage	A kernel bug	Platform-specific
4	Everything above, portably, plus a disposable filesystem	Kernel exploits, host mounts, the docker socket	A Dockerfile
5	Most kernel exploits; VM-escape-level separation	Misconfiguration	Runtime install + ops
6	Everything, including your host existing	Latency, network dependency, cost	An API key

> The useful rule: climb only as far as the unattended case requires. Tiers 1–2 are worth having even at tier 4, because they fail with a readable error instead of a container startup cost — and they are what the tests can exercise offline.

03 · Tier 0 — the prompt

Tier 0 is what most agent demos ship: a system prompt that says *work inside the sandboxed folder*, plus an approval prompt on commands. It is worth naming as a tier rather than skipping, because it is not nothing — and because its failure mode is instructive. A model that has read a thousand shell tutorials will eventually `cd ..`, not out of malice but because it is trying to be helpful.

On the day the model is helpful in the wrong direction, tier 0 turns a demo into an incident. The rest of this page is the cheap insurance.

> Language matters here. Calling the folder a “sandbox” when only the prompt enforces it is how a demo becomes a cautionary tale. Either the mechanism is there or the claim is not.

04 · Tier 1 — containment in code

This is the rung the repo is already on, and the first one worth writing because it costs nothing and its tests never touch the network. Every path is resolved by one function, so there is exactly one place to be right — and `src/sandbox.js` wraps it for the three actions that change a file (`save`, `edit`, `delete`), printing one bold red `FORBIDDEN` line per refusal. Reads go through the same resolver without the containment check: reads are open, writes are not.

SRC/WORKSPACE.JS — THE CONTAINMENT CHECK · *javascript*

```
resolve(requested) {
  const raw = requested.trim();
  const expanded = expandVars(raw);                      // ~ and %VAR% and $VAR

  if (/^[A-Za-z]:/.test(raw))      throw new PathError(`absolute or drive-relative path is outside the workspace:
${raw}`);
  if (/^(\\\\|\/\/)/.test(raw))    throw new PathError(`UNC path is outside the workspace: ${raw}`);
  if (path.isAbsolute(expanded))   throw new PathError(`absolute path is outside the workspace: ${raw}`);

  const first = raw.split(/[\\/]/)[0].toLowerCase().replace(/:$/, '');
  if (/^[~$%]/.test(first) && first !== raw)
    throw new PathError(`home/environment path is outside the workspace: ${raw}`);

  const abs  = path.resolve(this.root, expanded);
  const real = realResolve(abs);                         // deepest existing ancestor, realpath'd
  if (!isInside(this.root, real)) throw new PathError(`path escapes the workspace: ${raw}`);

  return { abs: real, rel: path.relative(this.root, real) };
}
```

Two design choices are worth copying whatever tier you land on. First, the check throws a typed `PathError` that the dispatcher turns into an `ERROR:` string, so a refusal is an observation the model can read rather than a crash. Second, it refuses *before* resolving, so the error message describes the request instead of a half-computed path.

SRC/WORKSPACE.JS — WHY A STRING CHECK IS NOT ENOUGH · *javascript*

```
function realResolve(abs) {
  // realpath the deepest ancestor that exists, then re-append the tail
  let p = abs, tail = '';
  while (!fs.existsSync(p)) {
    const parent = path.dirname(p);
    if (parent === p) break;
    tail = path.join(path.basename(p), tail);
    p = parent;
  }
  return path.join(fs.realpathSync.native(p), tail);
}
```

> What tier 1 buys: a model that hallucinates ../../../.env gets a sentence back and tries again. What it does not buy: a shell. run_command has no path to check because a shell is not a path — the next rung is where commands get bounded, and even then only the writes you can read off the command line (copy , del , a redirect) are refused.

05 · Tier 2 — process hardening

The next rung is still ordinary code, and it is where most of the real-world value is. Four things, none of them exotic: a deny-list, a timeout, an output cap, and an environment scrub.

SRC/TOOLS.JS — THE HARD-DENY LIST, CHECKED BEFORE POLICY · *javascript*

```
const HARD_DENY = [
  { re: /rm\s+-rf\s+[\/~]/i,           why: 'recursive delete of a root path' },
  { re: /\bdel\s+\/[sq]\b/i,            why: 'recursive or quiet Windows delete' },
  { re: /\b(format|diskpart)\b/i,       why: 'disk formatting' },
  { re: /\b(shutdown|reboot)\b/i,       why: 'machine power control' },
  { re: /git\s+push\s+(-f|--force)/i,   why: 'force push' },
  { re: /(curl|wget)[^|]*\|\s*(sh|bash)/i, why: 'piping the internet into a shell' },
  { re: /:\(\)\s*\{.*\|.*\};:/,         why: 'fork bomb' },
  { re: />\s*\/dev\/sd[a-z]/i,          why: 'raw disk write' },
];
```

A deny-list is a seatbelt, not a boundary: it catches the eight ways a model ruins a demo, and it catches nothing that was written to get past it. That is fine at this rung, as long as nobody calls it isolation.

The three bounds around a spawned process

SRC/TOOLS.JS — BOUNDED EXECUTION (CONDENSED) · *javascript*

```
function runShell(command, cwd, { timeoutMs = 30000, maxBytes = 64 * 1024 } = {}) {
  return new Promise((done) => {
    const child = spawn(command, { cwd, shell: true, windowsHide: true,
                                   env: safeEnv(),          // <- see below
                                   detached: process.platform !== 'win32' });

    let out = '', err = '', truncated = false;
    const cap = (buf) => {
      const s = buf.toString();
      if (out.length + err.length + s.length > maxBytes) { truncated = true; return; }
      return s;
    };
    child.stdout.on('data', (d) => { const s = cap(d); if (s) out += s; });
    child.stderr.on('data', (d) => { const s = cap(d); if (s) err += s; });

    const timer = setTimeout(() => killTree(child), timeoutMs);
    child.on('close', (code) => {
      clearTimeout(timer);
      done({ code, stdout: out + (truncated ? '\n[output truncated]' : ''), stderr: err });
    });
  });
}
```

> The environment is the leak nobody notices. A child process inherits process.env , which holds OPENROUTER_API_KEY . One echo $OPENROUTER_API_KEY puts the key into messages , which goes back to the provider and into your logs. safeEnv() should hand the child a short allow-list — PATH , HOME , TMPDIR , nothing else.

THE TWO FUNCTIONS THAT MAKE THE BOUNDS REAL · *javascript*

```javascript
function safeEnv() {
  const keep = ['PATH', 'PATHEXT', 'HOME', 'USERPROFILE', 'TMPDIR', 'TEMP', 'TMP', 'SYSTEMROOT', 'COMSPEC'];
  const env = {};
  for (const k of keep) if (process.env[k] !== undefined) env[k] = process.env[k];
  return env;
}

function killTree(child) {
  if (process.platform === 'win32') spawn('taskkill', ['/pid', String(child.pid), '/t', '/f']);
  else process.kill(-child.pid, 'SIGKILL');       // the negative pid hits the group
}
```

`killTree` matters more than it looks: a 30-second timeout that kills only the shell leaves `node` or `npm` running, and a demo with three orphans on port 5000 is a demo that “mysteriously” stopped working.

06 · Why hardening is not isolation

Everything in tiers 1–2 is advice the process gives itself. The process can also ignore it, because the model does not run in the harness — it emits a string, and the harness hands that string to a shell that has all of the harness's privileges.

A single command is enough to show the gap:

ONE COMMAND, NO PATH CHECK, NO CONTAINER · *shell*

```
cd /c/Users/you && ls -la .ssh && cat .ssh/id_rsa | head -3
```

There is no `resolve()` call anywhere in that line. `run_command` starts in `workspace/` and the first thing a shell can do is leave. Tier 2 reads the *write targets* off the command line (`copy`, `del`, `>`, `-Destination`) and refuses the ones outside, but that is a scan of the arguments, not a boundary: `node -e`, a Python one-liner or a downloaded script can still reach anything the process can reach. Pretending otherwise is the mistake this page exists to prevent.

PROPERTY	TIER 1	TIER 2
Can write outside the workspace	via file tools: no	via file tools: no
Can write outside the workspace	via `run_command`: for the commands it can read (`copy`, `move`, `del`, `>`) no; for a script it runs, yes	same: a command-line scan, not isolation
Can read `~/.ssh`	yes, through the shell	yes, through the shell
Can reach the network	yes	yes (`curl` blocked, `node -e` not)
Can see the API key	yes	no, with `safeEnv()`
Can run for ten minutes	no	no, 30 s timeout

> So why bother? Because these two rungs make the common case fail loudly and cheaply, and they keep working after you add a container. A refusal that costs one loop step is better than a container start that costs two seconds.

07 · Tier 3 — operating system

The next rung uses the thing that already exists for this: the operating system. The idea is one sentence — run the command as a different user, in a different namespace, with no network — and the implementations are wildly platform-specific.

Linux: namespaces in one command

BUBBLEWRAP (THE FLATPAK SANDBOX) — ROOTLESS, NO DAEMON · *shell*

```
bwrap --unshare-all --die-with-parent \
  --ro-bind /usr /usr --ro-bind /lib /lib --ro-bind /lib64 /lib64 \
  --bind "$PWD" /work --chdir /work \
  --tmpfs /tmp --proc /proc --dev /dev \
  --new-session -- /bin/sh -lc "$COMMAND"
```

Swap `bwrap` for `nsjail` when you also want a seccomp filter, a uid map and a per-run network namespace, or for `unshare --user --map-root-user --net --mount` when you want to write it yourself and debug it. This is the lightest real isolation available on Linux: single-digit milliseconds, no image to build, and the flags *are* the policy, which makes it auditable.

Windows: no equivalent, and that is the finding

MECHANISM	WHAT IT GIVES	REALITY FOR THIS PROJECT
`runas` / a second local user	a different uid for file permissions	needs a password prompt; file ACLs must actually deny the workspace to the other user
Job objects	memory and CPU caps, kill-on-close	good for bounds, gives no filesystem or network isolation
AppContainer / restricted token	the real per-app sandbox WinRT uses	not reachable from Node without native code or a helper binary
Windows Sandbox	a disposable desktop VM	Pro/Enterprise only, tens of seconds to start, awkward to script per-command
WSL2 + Docker	a Linux kernel in a VM	the practical answer on Windows, which is why the next tier is Docker

> Pragmatic conclusion: on Windows, tier 3 is either “add bounds and accept the gap” or “go straight to Docker”. Nobody writes an AppContainer wrapper for a meetup demo, and the people who do are writing it for a product.

08 · Tier 4 — Docker

This is the rung most teams land on, and the one this page spends its time on, because it is the first that gives you a disposable filesystem *and* a network namespace *and* works the same on Windows, macOS and Linux — once Docker Desktop is installed.

The whole design fits in one `docker run`. The flags matter more than the image:

THE FLAG SET THAT TURNS A CONTAINER INTO A SANDBOX · *shell*

```
docker run --rm --init \
  --network none \
  --read-only --tmpfs /tmp:rw,noexec,nosuid,size=64m \
  --memory 512m --memory-swap 512m --pids-limit 128 --cpus 1 \
  --cap-drop ALL --security-opt no-new-privileges \
  --user 65534:65534 \
  --mount type=bind,source="$PWD/workspace",target=/work \
  -w /work \
  node:20-alpine sh -lc "node demo/hello.js"
```

FLAG	WHAT IT PREVENTS
`--network none`	Exfiltration, `npm install` at run time, and a model that decides to fetch a package it half-remembers.
`--read-only` + `--tmpfs /tmp`	Writes anywhere except the mount and a 64 MB scratch dir. Deleting the container deletes them.
`--memory`, `--memory-swap`	A runaway allocation that takes the laptop with it.
`--pids-limit`	Fork bombs, and `npm`'s habit of spawning a process tree.
`--cpus 1`	A hot loop that makes the demo machine unusable.
`--cap-drop ALL`	The handful of capabilities root-in-container gets by default.
`--security-opt no-new-privileges`	Regaining privilege through a setuid binary.
`--user 65534:65534`	Files in the mount being written as your user — `nobody` has to be given permission deliberately.
`--rm --init`	Leftover containers, and zombie children holding the image lock.

> The one flag you must never add. Do not mount /var/run/docker.sock into the sandbox. It is not a file, it is root on the host: anything with the socket can start a privileged container and mount / .

Wiring it into the harness

The pleasant surprise is how little changes. `run_command` already returns `{ code, stdout, stderr }`, so the container version is the same function with a different `spawn` target — the timeout, the output cap and the tool result string all stay:

SRC/SANDBOX/DOCKER.JS — ILLUSTRATIVE SKETCH, NOT YET IN THE REPO · *javascript*

```
export function dockerSandbox(root, image = 'node:20-alpine') {
  return {
    name: `docker:${image}`,

    async exec(command, { timeoutMs = 30000 } = {}) {
      const args = [
        'run', '--rm', '--init',
        '--network', 'none',
        '--read-only',
        '--tmpfs', '/tmp:rw,noexec,nosuid,size=64m',
        '--memory', '512m', '--memory-swap', '512m',
        '--pids-limit', '128', '--cpus', '1',
        '--cap-drop', 'ALL',
        '--security-opt', 'no-new-privileges',
        '--user', '65534:65534',
        '--mount', `type=bind,source=${root},target=/work`,
        '-w', '/work',
        image, 'sh', '-lc', command,
      ];
      return await runBounded('docker', args, { timeoutMs });   // same spawn + killTree as tier 2
    },
  };
}
```

Two details earn their keep. `runBounded` is the tier-2 function with the shell swapped for an argv array, so escaping stops being a quoting problem. And the mount is the *only* writable host path, which means the model can still create files your editor can open — the demo stays a demo.

09 · What Docker does not give you

A container is a process with a convincing costume: the same kernel, a different view of it. That view is strong enough for the threat model in chapter 01 and weak enough that it must be described accurately.

CLAIM	REALITY
"The container cannot see my files"	True for everything not mounted. The bind mount is deliberate and is the escape hatch you built.
"The container cannot escape"	It shares the host kernel. A kernel privilege-escalation bug is a container escape; that is why tier 5 exists.
"It runs as `nobody` so it cannot hurt anything"	True until something in the mount is owned by `nobody`, or a setuid binary creeps into the image.
"The code inside is safe to run"	The image is code you pulled from a registry. Pin by digest and rebuild it yourself, or the sandbox's supply chain is the model's supply chain.
"Docker Desktop on Windows is a Linux host"	It is a VM. That is *better* for you — the host filesystem is not the container's kernel neighbour — but the mount is still a real host folder, and it is slow.
"A symlink written in the mount is harmless"	The container can create `workspace/x -> C:/Users/you/.ssh`. Host-side file tools must still call `resolve()`, which realpaths the link and refuses it. Container isolation does not replace the check; it protects the shell.

> Keep tier 1 running underneath tier 4. The container guards the shell; Workspace.resolve() guards the file tools, including anything the container left behind in the shared folder. They cover different holes.

The costs nobody mentions in the slide

- **Image pull is the real start time.** A warm local image starts in a fraction of a second; the first run on a fresh laptop pulls hundreds of megabytes over the meetup wifi, in front of the room.
- **Docker Desktop is another failure surface.** A demo that now has two ways to fail (the model, and the daemon) needs a fallback: `--sandbox local` should still work.
- **Bind-mount permissions on Windows** map differently through the VM, so files land as a uid the host does not have. `chmod` inside is cosmetic; plan for it or the `npm test` inside the container fails for a reason unrelated to the agent.
- **Networking is off, so nothing installs.** If the task needs a dependency, bake it into the image or mount a `node_modules` — the model cannot invent one at run time, which is a feature you should say out loud.
- **Startup latency lands inside the loop.** One container per command is correct and slow (~0.5 s each). One long-lived container with `docker exec` is fast, and now the command has state — a shell that remembers `cd`, and files that persist between steps. That is usually what you want; it is a different design, and it needs a `stop()`.

10 · Tier 5 — stronger kernel isolation

Tier 5 is for the day a container is not enough: multi-tenant, untrusted code, or a security review that asks what happens when the kernel is wrong. There are three shapes.

b += table(["Shape", "Example", "What changes"], [["User namespaces", "rootless Docker, `userns-remap`, Podman", "Root in the container is not root on the host. Cheapest real hardening: same `docker run`, though it is broken on some CI runners."], ["Userspace kernel", "gVisor (`--runtime=runsc`)", "Syscalls are intercepted and re-implemented in Go, so a kernel bug becomes a gVisor bug. Costs a few percent to a few times on syscall-heavy work, which `npm install` is."], ["Own kernel", "Kata Containers, Firecracker microVMs", "A hardware virtualisation boundary per sandbox. Firecracker's own figures are ~125 ms boot and a few MB of overhead, which is why it underpins the hosted sandboxes in chapter 11."],])

THE SWAP IS ONE FLAG, IF YOU BUILT THE INTERFACE · *shell*

```shell
# same interface, different runtime — the code above does not change
docker run --runtime=runsc ...            # gVisor
docker run --runtime=kata-runtime ...     # Kata microVM
```

> Verdict for a meetup demo: tier 5 is where you stop and say “this is a product decision”. The interesting content for the room is that the interface exists, so the runtime is a flag rather than a rewrite.

11 · Tier 6 — someone else's machine

Two ways to not run the sandbox yourself. They look similar and solve different problems.

Hosted sandboxes: an API call per run

E2B-SHAPED API — MODAL, DAYTONA AND CLOUDFLARE'S SANDBOX SDK ARE VARIATIONS · *javascript*

```javascript
import { Sandbox } from 'e2b';

const sbx = await Sandbox.create({ template: 'node20' });     // ~1 s cold, faster warm
try {
  await sbx.files.write('/work/hello.js', source);
  const res = await sbx.commands.run('node hello.js', { timeoutMs: 30000 });
  console.log(res.exitCode, res.stdout, res.stderr);
} finally {
  await sbx.kill();
}
```

GAIN	GIVE UP
Isolation is their problem, and they take it more seriously than you will	Every command is a network round trip; the demo now needs wifi
Per-run VM or microVM, so state cannot leak between users	Your workspace lives on someone else's disk, and so does the code
Horizontal scale: one sandbox per task, ten at once	Latency floor of a few hundred ms per call, plus a vendor SDK in the tool handler
No Docker Desktop on the demo laptop	An account, a key, and a quota — three new ways for the talk to fail

> The honest fit: tier 6 is right for a product that runs untrusted code for strangers. For one laptop and one audience it is more moving parts than the risk justifies — but it is the right answer to "how would you ship this?" and worth one slide.

WASM: a sandbox for expressions, not for programs

If the untrusted thing is a *computation* rather than a coding agent, a WebAssembly runtime is the cheapest real boundary there is: no syscalls to intercept, an in-memory filesystem, no network, and microsecond start-up. QuickJS or Pyodide in a Node worker gets you a scripting sandbox in a few dozen lines.

It is not a coding sandbox. There is no `npm`, no native binary, no long-running server, no `git`, and no `spawn` — which is exactly the list a coding agent needs. File it under "the other tool you might reach for", not under "the answer".

12 · Wiring it into the agent

Every tier above changes what happens inside `run_command` and almost nothing else. That is the argument for defining the seam early: one module, one contract, three implementations, chosen by config.

THE INTERFACE THAT MAKES THE LADDER A CONFIG OPTION · *javascript*

```
// src/sandbox/index.js — everything the tool layer is allowed to know about isolation
//
//   { name, exec(command, { timeoutMs }) -> { code, stdout, stderr }, stop() }

export function createSandbox(mode, { root }) {
  switch (mode) {
    case 'local':  return localSandbox(root);    // tiers 1–2, today
    case 'docker': return dockerSandbox(root);   // tier 4
    case 'remote': return e2bSandbox(root);      // tier 6
    default: throw new Error(`unknown sandbox mode: ${mode}`);
  }
}
```

THE DIFF, IN FULL · *javascript*

```
// src/tools.js — before
const { code, stdout, stderr } = await runShell(command, ws.root);

// src/tools.js — after
const { code, stdout, stderr } = await ctx.sandbox.exec(command, { timeoutMs: 30000 });
```

Three things to keep in mind when you do this refactor.

- **Keep `HARD_DENY` and `CMD_POLICY` outside the sandbox.** They are policy about the model, not capability of the runtime. A container does not make `git push --force` acceptable.
- **Let the file tools stay host-side at tier 4** — the mount makes it work, and `resolve()` keeps working. Only at tier 6 do reads and writes become RPC, and that is when the tool handlers genuinely change shape.
- **Make the mode a startup banner line.** `sandbox: docker (network none, 512 MB)` printed once tells the audience, and tells you in a bug report, which tier is actually running.

> Name the degradation explicitly. If Docker is missing, do not silently fall back to local . Print sandbox: local (no isolation for commands) and carry on — a visible downgrade is safe, a silent one is how the wrong thing ships.

13 · Red-team it

A sandbox you have not attacked is a sandbox you are guessing about. These are the checks that separate “we configured it” from “we tried it”, and they double as offline tests for tiers 1–2.

ATTEMPT	TIER 1–2	TIER 4	TIER 6
`cat ~/.ssh/id_rsa`	succeeds	no such file	no such file
`echo $OPENROUTER_API_KEY`	leaks, unless `safeEnv()`	empty	empty
`curl example.com`	succeeds	no network	usually blocked
`cd .. && ls`	your project root	container root	sandbox root
`dd if=/dev/zero of=/tmp/x`	fills the disk	tmpfs full at 64 MB	capped
`:(){ :|:& };:`	machine stalls	pids-limit kills it	capped
`ln -s ~/.ssh x`	shell follows it	host `resolve()` still refuses	n/a
`sleep 600`	killed at 30 s	killed at 30 s	killed at 30 s
`yes` forever	cut at 64 KB	cut at 64 KB	cut at 64 KB

THE THREE THAT CATCH THE REGRESSIONS · *javascript*

```javascript
// test/sandbox.test.js — the same table, as assertions (no Docker needed for the top half)
test('refuses a path that escapes the workspace', () => {
  assert.throws(() => ws.resolve('../../secrets.txt'), /escapes the workspace/);
});
test('a symlink out of the workspace is refused by its real target', () => {
  fs.symlinkSync(os.homedir(), path.join(root, 'link'));
  assert.throws(() => ws.resolve('link/.ssh'), /escapes the workspace/);
});
test('the environment handed to a command holds no keys', () => {
  assert.deepEqual(Object.keys(safeEnv()).filter(k => /KEY|TOKEN|SECRET/.test(k)), []);
});
```

14 · Which rung, when

The short version, for the demo and for the talk.

SITUATION	DO THIS	WHY
The meetup demo, you at the keyboard	tiers 1–2, `CMD_POLICY=ask`	The human is the control, and the loop is the thing you are showing.
Unattended run on your own machine	tier 4, `--yes`, `--network none`	The container bounds the worst case; the deny-list still stops the obvious.
CI, or a second contributor	tier 4, one container per task	Reproducible: the image is the environment, and nothing secret is mounted.
Running other people's code	tier 5 or 6	Kernel boundary or their machine. Container isolation is not enough for strangers.

And the two sentences worth saying out loud in the room, because they are the whole talk:

> 1. The sandbox is not the security model — the interface is. Once run_command goes through one function, isolation becomes a flag, and you can climb the ladder without rewriting the agent. 2. Every tier below the one you need is a bug report waiting to happen; every tier above it is a demo that will not start. Tier 4 is the default because it is the cheapest rung that survives being left alone.

Then demo the failure, not the success. Ask the agent to `cat .env`, let the refusal come back as a tool result, and watch it try another way — that is the loop doing its job, and the sandbox doing its.